

REPORT FOR THE MINI-PROJECT IoT

Hospital Room and Patient Monitoring System

Realized by:

Ahmed CHAABANE
Firas KAHLAOUI
Seifeddine BEN FRADJ

Contents

1	Introduction	1
1	Context	1
2	Problem Statement	1
3	Objectives	2
4	Contributions	2
5	Organization	3
2	Problem Analysis and Preliminary Study	4
1	System Description	4
2	Use Cases	5
3	Inputs and Outputs	6
4	Constraints	7
3	System Architecture	10
1	Global Architecture Diagram	10
2	Functional Architecture	10
3	Hardware Architecture	12
4	Cloud and Server Infrastructure	12
5	Software Architecture	13
6	Processing Pipeline	15
4	Methodology	17
1	System Design	17
1.1	Embedded Subsystem (ESP32)	17
1.2	Backend Subsystem Methodology	18
1.3	Web Dashboard Subsystem Methodology	19
1.4	AI Subsystem Methodology	19
1.5	Face Recognition Methodology	20
1.6	Alert Stabilization and Rate Limiting	20
1.7	Algorithmic Pseudocode	20
2	Methodology Diagram	21

5	Implementation	23
1	System Overview	23
2	Hardware Implementation	24
3	Hardware Implementation Challenges	24
4	Hardware Schematics	25
5	Software Development Environment	25
6	Software Implementation	26
6.1	Embedded Subsystem	26
6.2	Backend Implementation	27
6.3	Web Dashboard Implementation	27
7	Software Validation	27
7.1	Embedded Subsystem Validation	27
7.2	Backend Validation	28
7.3	Web Dashboard Validation	28
8	Embedded Implementation (ESP32)	28
8.1	Memory Optimisation	28
8.2	Latency Optimisation	28
9	AI Integration	29
6	Experimental Results and Validation	30
1	Experimental Protocol	30
1.1	Embedded Subsystem Protocol	30
1.2	Backend Protocol	30
1.3	Web Dashboard Protocol	30
1.4	AI Protocol	30
2	Software Results (PC / Server)	31
3	Embedded Results (ESP32)	31
4	Latency and FPS Analysis	32
5	Comparison: Design Specification vs. Observed Behaviour	33
5.1	Specification vs. Implementation (Embedded Subsystem)	33
5.2	Theoretical Model vs. Observed System Behaviour	33
6	Error Analysis	34
6.1	Embedded Subsystem	34
6.2	Backend, Web Dashboard, and AI Subsystems	34
7	Discussion	35
1	Validity of Results	35
1.1	Embedded Subsystem	35
1.2	Backend and AI Subsystems	35

1.3	Web Dashboard Subsystem	35
2	Limitations	35
2.1	Embedded Subsystem	36
2.2	Backend Subsystem	36
2.3	Web Dashboard Subsystem	36
2.4	AI Subsystem	37
3	ESP32 Constraints Impact	37
4	Robustness	37
5	Identified Improvements	37
8	Conclusion	39
1	Summary	39
2	Achievement Against Objectives	39
3	Limitations	40
4	Future Work	40
9	AI Predictive Analytics	42
1	Problem Formulation	42
2	Statistical Anomaly Detection	42
3	Predictive Forecasting	42
4	Clinical Interpretation Engine	43

List of Figures

2.1	System context and operational boundaries.	5
3.1	Global architecture of the system.	10
3.2	Functional decomposition and inter-subsystem interfaces.	11
3.3	System-wide software architecture and module boundaries.	14
3.4	Persistence layer schema: Firestore collections and RTDB paths.	14
3.5	Service layer orchestration and API architecture showing tRPC procedures and middleware.	14
3.6	Real-time data processing and alerting sequence from sensor to dashboard.	16
4.1	Face recognition pipeline and clinical alert decision flow.	19
4.2	Overall development methodology.	22
5.1	Backend service and data integration overview showing tRPC/Firebase interaction.	23
5.2	Hardware wiring diagram	25
6.1	System-wide validation pipeline from hardware to cloud reporting.	31
6.2	ESP32 state machine and Wi-Fi/Firebase communication flow.	32
9.1	AI predictive analytics pipeline: from vitals window to status synthesis. .	43

List of Tables

3.1	ESP32 firmware module summary.	13
5.1	ESP32 software development environment.	26
5.2	Full-stack software development environment.	26
6.1	System stability and recovery validation results.	31
6.2	Analytical timing bounds derived from implementation constants.	32
6.3	Specification vs. implementation validation for the embedded subsystem.	33
6.4	Specification vs. implementation mapping for backend and web.	33

Liste des acronymes

IOT: Internet of things.

AI: Artificial intelligence.

HTTPS: Hypertext Transfer Protocol Secure.

BPM/ Beat per minute.

tRPC: TypeScript Remote Procedure Call.

NVS: Non-Volatile Storage.

RBAC: Role-Based Access Control

SMTP: Simple Mail Transfer Protocol.

FIFO: First in first out.

JSON: JavaScript Object Notation.

API: Application programming interface.

PPG: Photoplethysmogram.

MSE: Mean squared error.

MAE: Mean average error.

RMSE: Root mean squared error.

NaN: Not a number.

UI: User interface.

JWT: JSON web token.

NVS: Non-Volatile Storage.

Chapter 1

Introduction

1 Context

The healthcare sector is undergoing a significant transformation driven the Internet of Things (IoT) technologies. Continuous, real-time monitoring of patient vital signs and ambient room conditions is a foundational requirement in any clinical environment, enabling medical staff to detect deterioration early, respond rapidly, and reduce the administrative overhead associated with manual observation rounds.

This project proposes a complete, end-to-end hospital monitoring solution composed of four integrated subsystems:

- **Embedded acquisition node (ESP32):** real-time sensor acquisition and authenticated cloud transmission.
- **Backend server:** database management, alert rule evaluation, and user authentication.
- **Web dashboard:** real-time data visualisation, historical charts, and alert management for medical staff.
- **AI insights layer:** statistical anomaly detection and short-horizon trend forecasting executed on the server and exposed through a typed API.

2 Problem Statement

Hospital rooms require continuous monitoring of two categories of parameters: environmental conditions (temperature and humidity), which directly affect patient comfort, infection risk, and equipment operation; and patient vital signs (heart rate and blood oxygen saturation), which are primary indicators of physiological status.

The current approach in many low-resource clinical settings relies on periodic manual measurement, which introduces observation gaps and increases nursing workload.

3 Objectives

The project objectives are divided into functional and technical categories.

Functional Objectives

1. Continuously acquire temperature ($^{\circ}\text{C}$), humidity (%), heart rate (BPM), and blood oxygen saturation (SpO_2 , %) from dedicated sensors.
2. Transmit sensor data to a cloud database at a near-real-time rate of up to 1 Hz.
3. Implement a web dashboard for real-time data visualisation, history browsing, and alert management.
4. Generate automated predictions and anomaly alerts via a cloud AI model.

Technical Objectives

1. Availability: the system shall operate continuously with network recovery mechanisms.
2. Reliability: invalid or inconsistent data shall be detected and flagged.
3. Security: secure authentication and encrypted communication (HTTPS).
4. Performance: near real-time data updates based on sampling frequency.

4 Contributions

This project is the result of a full-stack collaborative effort across four subsystems. The concrete contributions of the group are as follows:

Embedded acquisition node (ESP32). Complete firmware design and implementation: dual-core FreeRTOS task architecture, DHT22 and MAX30102 sensor drivers, PPG-based heart rate and SpO_2 computation, delta-threshold upload filter, LittleFS-backed configuration portal, NVS credential persistence, and automatic Wi-Fi recovery. Hardware assembly including resolution of the MAX30102 pull-up resistor incompatibility.

Backend server. Design and deployment of a Node.js/tRPC service layer with Firebase Admin SDK integration, Role-Based Access Control (RBAC), Zod schema validation, event and alert log generation for presence monitoring, a statistical AI insights endpoint for vitals, and an SMTP email procedure triggered by dashboard alerts.

Web dashboard. Implementation of a React/Vite single-page application with real-time Firebase RTDB subscriptions, Recharts-based vital sign visualisation, an AI Insights panel, and an on-device face-api.js pipeline running in the browser for authorized personnel tracking using an external USB camera.

AI insights layer. Server-side implementation of a statistical anomaly detection engine (Z-score over a short sliding window of recent points, with a minimum of 5 samples) and a 5-step vital sign forecasting model (Ordinary Least Squares linear regression), exposed as tRPC procedures and integrated into the dashboard's clinical status display.

5 Organization

The remainder of this report is structured as follows. Chapter 2 provides a detailed problem analysis, defining use cases, system inputs and outputs, and platform constraints. Chapter 3 presents the global and functional system architecture, including the hardware and software decomposition. Chapter 4 describes the design methodology, covering the mathematical models, pseudocode for the main algorithms, and the development workflow. Chapter 5 details the implementation of all subsystems, including hardware assembly challenges, software modules, and validation procedures. Chapter 6 reports the experimental results and quantitative validation for each subsystem. Chapter 7 discusses the validity of the results, system limitations, and identified improvements. Chapter 8 concludes the report with a synthesis of achievements and directions for future work. Chapter 9 provides an expanded treatment of the AI prediction layer, including its quantitative evaluation and performance constraints.

Chapter 2

Problem Analysis and Preliminary Study

1 System Description

The system is a networked, multi-tier hospital monitoring solution. At its base sits the **ESP32 embedded acquisition node**, which occupies the sensor layer and embedded layer. Its role is to collect physiological and environmental measurements, apply local signal processing to derive meaningful values (BPM, SpO₂), and transmit structured records to a cloud database.

The system follows a multi-tier architectural pattern designed for high availability and low-latency monitoring. At the edge, the **IoT Layer** (ESP32) performs continuous data acquisition. This telemetry is pushed to the **Cloud Persistence Layer** (Firebase), where Realtime Database handles live streams and Firestore manages long-term clinical records (users, people, alerts, and detection events). The **Service Layer** (Node.js/tRPC) provides a type-safe interface for management operations and AI inference, while the **Application Layer** (React) delivers a high-fidelity monitoring interface for medical staff. The face-recognition pipeline runs directly in the browser using face-api.js and feeds detection events back to the server for audit logging and alerting.

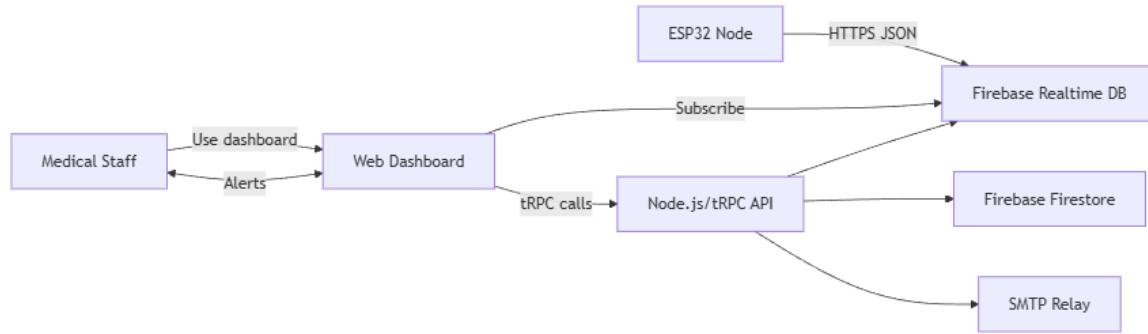


Figure 2.1: System context and operational boundaries.

2 Use Cases

The following operational scenarios define the expected system behaviour.

Embedded Node Use Cases

UC-1: Initial Configuration. A technician connects to the ESP32's Access Point and enter Wi-Fi credentials, Firebase credentials, room ID, and patient ID. The device saves all settings to NVS, serves a confirmation page, and reboots.

UC-2: Normal Operation. After boot, the device loads credentials from NVS, connects to the hospital Wi-Fi, authenticates with Firebase, and enters continuous monitoring mode.

UC-3: Wi-Fi Recovery. If Wi-Fi is lost during operation (or fails on cold start), the embedded device detects the disconnection and retries `wifiConnect()` every 10 seconds, indefinitely, without manual intervention.

UC-4: Remote Reconfiguration. A technician connects to the AP at any time to update credentials. After saving, the device reboots and applies the new configuration.

UC-5: Sensor Fault Detection. If a sensor returns an invalid reading (NaN from DHT22, or an IR raw value below the 50 000-count finger-presence threshold from the MAX30102), the firmware discards the sample, skips the upload, and continues monitoring without writing corrupt data to the database.

Backend Use Cases

UC-6: Staff Authentication and RBAC. Medical staff (Doctors and Nurses) authenticate via Firebase. The system enforces Role-Based Access Control (RBAC),

ensuring that clinical records are only accessible to authorized personnel assigned to the specific patient.

UC-7: Clinical Data Synchronization. Administrative users trigger synchronization between the local cache and the Firebase Firestore backend to ensure data consistency across multiple monitoring stations.

UC-8: Automated Alert Dispatch. The dashboard applies configurable thresholds to live telemetry; when a violation is detected, it triggers email alerts to the assigned Doctor and Nurse through the backend’s SMTP relay. The server also creates alert logs for abnormal presence events (unknown person, patient absence).

Web Dashboard Use Cases

UC-9: Real-time Telemetry Visualization. Staff view live vitals (BPM, SpO₂) and room conditions via high-frequency charts. The dashboard subscribes directly to the Realtime Database to ensure sub-second latency.

UC-10: Personnel Tracking via Face Recognition. Using the terminal’s camera, the dashboard identifies staff members entering the room and logs their presence. It cross-references detected faces against the authorized staff directory and flags unauthorized entries, applying a multi-frame stability requirement before issuing alerts.

AI Subsystem Use Cases

UC-11: Clinical Anomaly Detection. The AI layer analyzes time-series windows of patient vitals to identify statistical outliers (Z-score analysis) that may indicate sensor failure or acute physiological changes.

UC-12: Predictive Vital Forecasting. Using linear trend analysis, the AI subsystem projects the patient’s vitals for the subsequent 5 samples (e.g., ≈ 5 s at a 1 Hz input rate), providing clinicians with a short-horizon early-warning signal.

3 Inputs and Outputs

Embedded Node — Inputs

- **DHT22 (GPIO 4, one-wire):** digital temperature (°C) and relative humidity (%).
- **MAX30102 (I²C fast mode):** raw infrared and red photodiode FIFO values, used to compute BPM and SpO₂.

Embedded Node — Outputs

- **Firebase RTDB — /rooms/{roomId}**: JSON records containing `temperature`, `humidity`, and server-side `timestamp`, pushed on significant change.
- **Firebase RTDB — /patients/{patientId}**: JSON records containing `heartRate`, `spO2`, and server-side `timestamp`, pushed on significant change.

Identifier alignment. The dashboard subscribes to `/patients/{firebaseId}` for the selected patient. The ESP32 configuration therefore uses the patient's Firebase identifier as `patientId` to ensure consistent stream access across subsystems.

Backend, Web Dashboard, and AI — Inputs/Outputs

- **Backend Inputs:** tRPC procedure calls (JSON), Firebase ID Tokens, and Firestore/RTDB document snapshots.
- **Backend Outputs:** Structured clinical records (Users, People), alert logs, detection events, and SMTP-formatted email alerts when invoked by the dashboard.
- **Web Inputs:** Firebase RTDB telemetry stream, `face-api.js` descriptor vectors, and tRPC query results.
- **Web Outputs:** Real-time SVG/Canvas visualizations, detection event logs, and automated toast notifications.
- **AI Inputs:** Sliding-window vectors of Heart Rate and SpO2 values (target length 20; minimum 5 required for anomaly detection).
- **AI Outputs:** Rule-based clinical status (stable/warning/critical), anomaly flags, and 5-step trend forecasts.

4 Constraints

Memory Constraints

The ESP32 has 520 KiB of internal SRAM. The Firebase FreeRTOS task is allocated 8 192 bytes of stack, which must accommodate the TLS handshake context, async client buffers, and local variables. The NVS partition stores up to ten string key-value pairs for device configuration.

Latency Constraints

The Firebase upload interval is `FIREBASE_INTERVAL_MS = 1 000ms`, providing a near-real-time update rate. The DHT22 enforces a hardware minimum inter-sample interval of 2 000 ms.

Energy Constraints

The device is assumed to be mains-powered in a fixed hospital room installation. No active power management (light sleep, deep sleep, or radio duty cycling) is implemented in the current version.

ESP32 Platform Limitations

Sensor timing sensitivity. The MAX30102 heart rate and SpO_2 sensor produces a continuous stream of readings that must be drained from its internal buffer regularly. If the program is busy doing something else — such as waiting for a Firebase upload to complete, which can take several hundred milliseconds or more — the sensor buffer fills up and new samples are dropped. This results in the sensor reporting zero or no data, as if no finger were present, even when one is.

Why two cores are necessary. The ESP32 contains two independent processing cores (Protocol CPU and Application CPU) that can run tasks simultaneously. This hardware feature is the direct solution to the timing conflict described above. By assigning high-priority sensor acquisition exclusively to Core 1 and network-bound Firebase communication exclusively to Core 0, the system achieves **temporal isolation**. The sensor continues reading at its required frequency (draining the MAX30102 FIFO every 25 ms) regardless of how long the TLS handshake or Firebase upload takes. This ensures that no cardiac samples are dropped during network congestion, maintaining the integrity of the clinical data stream. Furthermore, Core 0's Wi-Fi management remains responsive, allowing for background SSID scanning and reconnection without introducing jitter into the physiological measurements.

AI and Cloud Constraints

The AI layer operates under **latency constraints** where inference must complete within the dashboard refresh cycle (15 s refetch for AI insights) and should remain well below the 1 Hz telemetry cadence. Computation is performed on the service layer to preserve the ESP32's limited SRAM for data acquisition. The anomaly model requires a minimum of 5 data points to compute a stable standard deviation.

Client-Side Compute Constraints

The face recognition pipeline executes in the browser and loads multiple pre-trained face-api.js models from a CDN. Model load time depends on bandwidth and browser caching; inference performance depends on WebGL availability and the client CPU/GPU. The detection loop is rate-limited to one frame every 600 ms to avoid saturating the UI thread on mid-range devices.

Data Model Constraints

The Firestore schema uses collections `users`, `people`, `detectionEvents`, `alertLogs`, and `roomActivityLogs`. The Realtime Database stores sensor streams under `/patients/{patientId}` and `/rooms/{roomId}`, and mirrors detection/alert logs under `/detectionEvents/{userId}` and `/alertLogs/{userId}` for real-time client subscription. A base64 storage fallback is used for face photos when Firebase Storage is not available.

Chapter 3

System Architecture

1 Global Architecture Diagram

Figure 3.1 presents the global architecture of the complete system, showing all five layers defined in the project specification: the sensor layer, the embedded layer, the server layer, the AI layer, and the application layer.

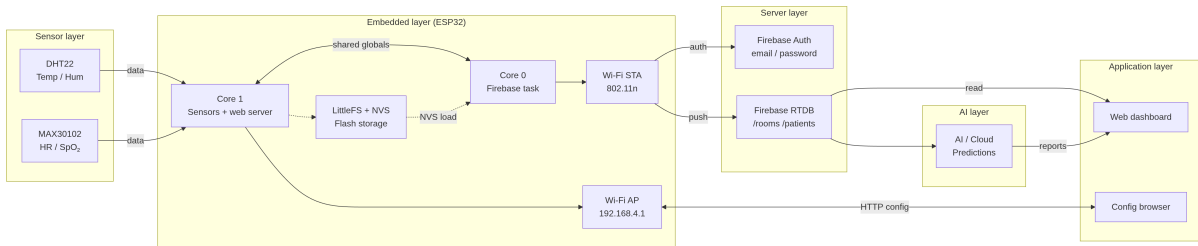


Figure 3.1: Global architecture of the system.

2 Functional Architecture

The system is functionally decomposed into four independent subsystems, each with a well-defined interface:

Network/Firebase Subsystem (ESP32, Core 0). Runs as a dedicated FreeRTOS task pinned to Core 0. Responsible for Firebase authentication (email/password over TLS), JSON serialization of sensor globals, HTTPS upload to the Realtime Database, and automatic Wi-Fi reconnection via `wifiConnect()`.

Sensor Subsystem (ESP32, Core 1). Executed on every iteration of the Arduino `loop()` on Core 1. Reads the DHT22 for temperature and humidity, drains the MAX30102 FIFO for raw PPG data, computes BPM via peak detection and SpO₂ via the ratio-of-ratios method, and writes results to shared float globals.

Settings/Web Server Subsystem (ESP32, Core 1). Registers HTTP route handlers for the configuration portal (served from LittleFS), handles form submissions to update Wi-Fi and Firebase credentials, persists all settings to NVS, and guards STA-side requests from accessing the configuration interface.

Configuration and Globals Layer. A header-only shared state layer (`globals.h`) that declares all inter-core shared variables, compile-time constants (upload interval, delta thresholds, IR presence threshold), and the DBG logging macros that compile to no-ops in release builds.

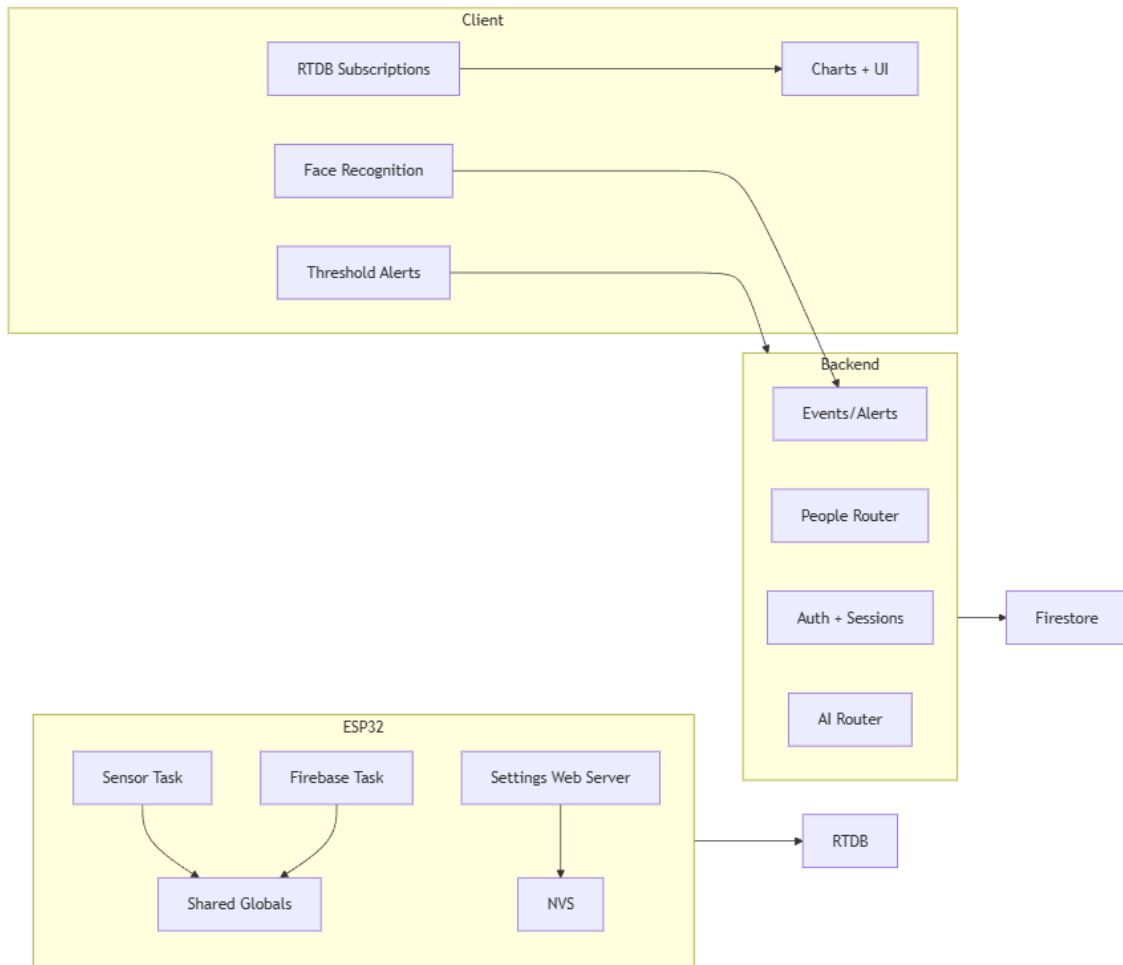


Figure 3.2: Functional decomposition and inter-subsystem interfaces.

Backend Functional Decomposition

The backend is architected as a **Modular tRPC Router** system. The **Authentication Middleware** establishes server sessions from Firebase ID tokens (login) and enforces session cookies for protected procedures. The **Clinical Management Layer** handles Firestore operations for users and people (patients/staff). The **Event and Alert Layer**

records presence events and creates alert logs, which are mirrored to the Realtime Database for real-time UI updates. The **AI Procedure Layer** provides windowed data analysis for predictive insights. SMTP email is available as an explicit procedure triggered by the dashboard when threshold alerts are raised.

Web and AI Functional Decomposition

The web dashboard is decomposed into four primary functional blocks: (1) the **Telemetry Subscription Bridge**, which maintains the high-frequency link to the Realtime Database; (2) the **Clinical Visualization Engine**, which manages the rendering of Recharts-based vitals; (3) the **Local Intelligence Module**, which executes face-recognition models in-browser and reports detection events every 600 ms; and (4) the **Alert Orchestrator**, which applies user-defined thresholds and triggers SMTP email alerts via a backend procedure. The AI subsystem decomposes into a **Statistical Signal Processor** for anomaly detection and a **Trend Extrapolator** for predictive forecasting.

3 Hardware Architecture

Microcontroller. ESP32-WROOM-32, dual-core at 240 MHz, 520 KiB SRAM, 4 MiB flash.

Temperature and Humidity Sensor. DHT22, single-bus digital interface on GPIO 4.

Pulse Oximeter and Heart Rate Sensor. MAX30102, I²C fast mode (400 kHz), SDA: GPIO 21, SCL: GPIO 22.

Communication. IEEE 802.11b/g/n Wi-Fi radio integrated on-chip. AP interface (192.168.4.1) serves the configuration portal.

4 Cloud and Server Infrastructure

The system is deployed on a **Serverless Cloud Infrastructure** to ensure high availability and scalability.

Compute Tier. The backend is hosted as a Node.js runtime environment on Vercel. This choice provides **automatic scaling** and **edge deployment**, reducing the latency of tRPC procedures for geographically distributed monitoring stations.

Persistence Tier. Firebase Firestore serves as the document-based repository. Its **flexible schema** allows for rapid iteration of clinical data models (e.g., adding face descriptors), while its **native integration** with Firebase Auth simplifies security rules.

Streaming Tier. Firebase Realtime Database is used for sensor telemetry. Unlike Firestore, RTDB is optimized for **frequent, small updates** and provides lower-latency push notifications via WebSockets, which is essential for sub-second clinical monitoring.

5 Software Architecture

Embedded Software (ESP32)

The firmware is organised into six source files following a strict, acyclic include hierarchy. Table 3.1 summarises the modules, their execution core, and their responsibilities.

Table 3.1: ESP32 firmware module summary.

File	Context	Responsibility
<code>globals.h</code>	Header	Shared variable declarations, DBG macros, timing constants
<code>sensors.h/cpp</code>	Loop	DHT22 and MAX30102 acquisition, BPM and SpO ₂ computation
<code>network.h/cpp</code>	FreeRTOS task	Firebase auth, upload, Wi-Fi management
<code>settings.h/cpp</code>	ISR callbacks	HTTP routes, LittleFS, NVS persistence
<code>esp32.ino</code>	Entry point	Global object definitions, <code>setup()</code> , <code>loop()</code>

The dependency graph is strictly acyclic: `esp32.ino` includes all modules; `network.h/cpp` and `settings.h/cpp` include `globals.h`; `sensors.h/cpp` includes `globals.h`. No module includes another module’s header directly.

Backend Software

The service layer is implemented as a **Node.js** application using the **trRPC** framework to provide a strictly typed API. This architecture eliminates the need for manual schema synchronization between the server and client. The backend utilizes the **Firebase Admin SDK** for secure authentication verification and uses **Firestore** as the primary system of record for users and people. Automated alert logic for presence events is driven by detection events logged from the dashboard, while telemetry threshold alerts are computed client-side and trigger email notifications through a dedicated procedure.

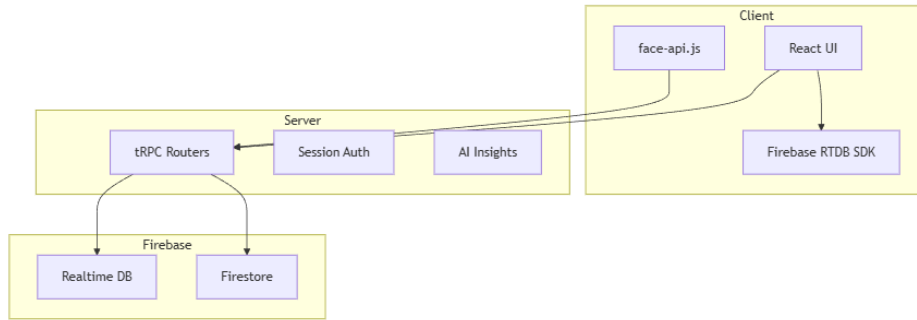


Figure 3.3: System-wide software architecture and module boundaries.

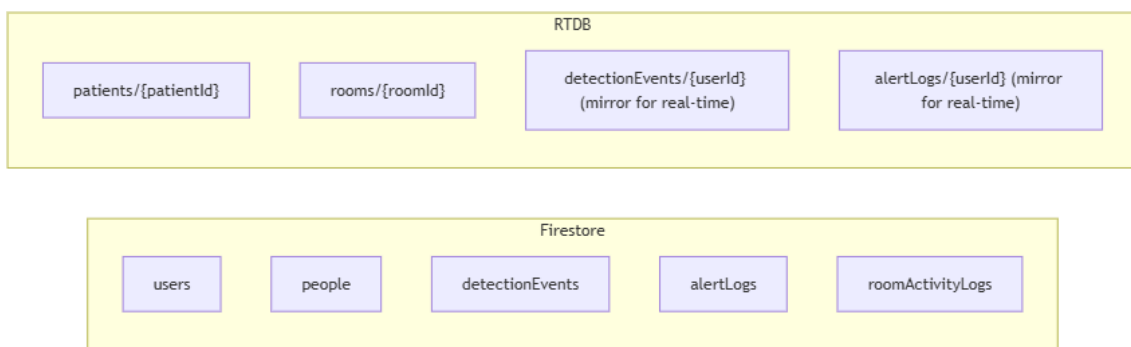


Figure 3.4: Persistence layer schema: Firestore collections and RTDB paths.

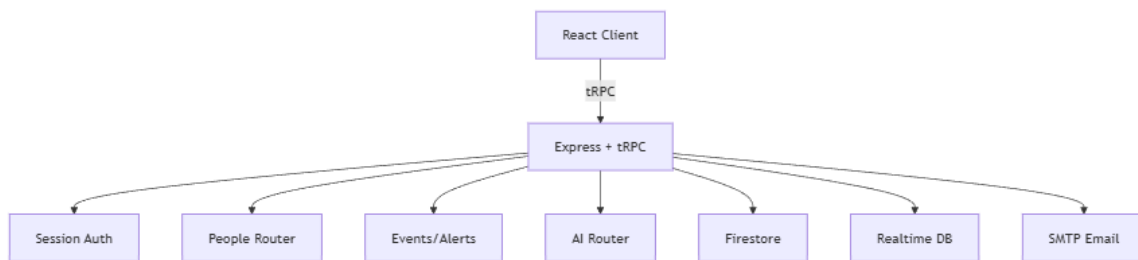


Figure 3.5: Service layer orchestration and API architecture showing tRPC procedures and middleware.

Web Dashboard Software

The user interface is a single-page application (SPA) built with **React** and **Vite**. It leverages the **Shadcn UI** component library for a modern, responsive clinical aesthetic. The application utilizes a reactive subscription model to the Firebase Realtime Database, ensuring that patient vitals are updated on-screen with sub-second latency. Data visualization is handled by the **Recharts** library, which renders interactive time-series

plots of the patient's physiological trends. Face recognition runs locally in the browser using face-api.js models loaded from a CDN and compares detected descriptors against enrolled identities stored in Firestore.

AI Software

The AI insights layer is integrated as a set of tRPC procedures that process windowed telemetry vectors. The system employs **statistical anomaly detection** using Z-score analysis to identify clinical outliers. For forecasting, the engine applies a **linear regression model** to project vital sign trends for the subsequent 5 samples (short-horizon trend). This predictive capability allows the dashboard to provide "AI Insights" that warn clinicians of potential instability before standard threshold alerts are triggered.

6 Processing Pipeline

The end-to-end data processing pipeline for a single sensor reading, from hardware to cloud record, proceeds as follows:

1. **Acquisition:** The ESP32 Core 1 samples the MAX30102 and DHT22 sensors.
2. **Processing:** Raw photoplethysmogram (PPG) data is processed via a peak-detection algorithm to derive heart rate and SpO2.
3. **Persistence:** Core 0 authenticates with Firebase and pushes the JSON-serialized vitals to the Realtime Database.
4. **Subscription:** The Web Dashboard, listening via WebSockets, receives the update and renders it in the live monitor.
5. **Inference:** The Dashboard periodically sends a 5–20 point history to the tRPC AI endpoint for anomaly analysis and forecasting.
6. **Alerting:** Threshold alerts are evaluated in the dashboard and dispatched through the SMTP relay; presence alerts are logged when detection events indicate unauthorized access or patient absence.

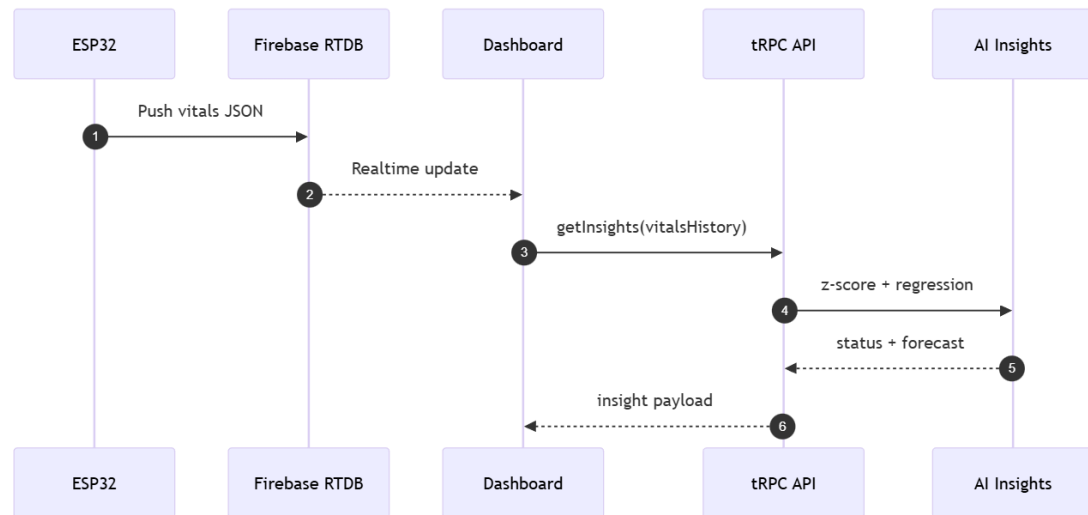


Figure 3.6: Real-time data processing and alerting sequence from sensor to dashboard.

Chapter 4

Methodology

1 System Design

1.1 Embedded Subsystem (ESP32)

1.1.1 Heart Rate

Heart rate is derived from the photoplethysmography (PPG) infrared signal by detecting peaks in the waveform — each peak corresponding to one cardiac cycle.

1.1.2 SpO₂

Blood oxygen saturation is estimated by pulse oximetry, exploiting the difference in optical absorption between oxyhaemoglobin (HbO₂) and deoxyhaemoglobin (Hb) at two wavelengths.

1.1.3 Delta-Threshold Upload Filter

Records are enqueued only when the measured value has changed by more than a defined threshold since the last successful push. Let x_n be the current reading and $\hat{x}$ the last transmitted value. The upload condition for room data is:

$$\text{Upload}_{\text{room}} \Leftrightarrow |T_n - \hat{T}| > \delta_T \vee |H_n - \hat{H}| > \delta_H \quad (4.1)$$

with $\delta_T = 0.3^\circ\text{C}$ and $\delta_H = 0.5\%$. For patient data:

$$\text{Upload}_{\text{patient}} \Leftrightarrow \text{BPM}_n \neq \hat{\text{BPM}} \vee |\text{SpO}_{2,n} - \widehat{\text{SpO}_2}| > \delta_S \quad (4.2)$$

with $\delta_S = 2\%$. Both conditions are evaluated every $T_{\text{interval}} = 1\,000\text{ ms}$.

1.1.4 End-to-End Latency Model

The total latency from sensor event to database record is:

$$L_{\text{total}} = T_{\text{acq}} + T_{\text{proc}} + T_{\text{serial}} + T_{\text{TLS}} + T_{\text{RTT}} \quad (4.3)$$

where:

- T_{acq} : sensor acquisition time (DHT22: up to 2 000 ms; MAX30102: continuous FIFO drain, ≈ 25 ms per sample at default rate).
- T_{proc} : local computation (< 5 ms per SpO_2 update; < 1 ms for BPM average).
- T_{serial} : JSON string build (< 1 ms).
- T_{TLS} : TLS handshake on first connection (1–3 s); session reuse on subsequent requests (< 5 ms).
- T_{RTT} : HTTPS round-trip to Firebase (≈ 100 – 300 ms over 2.4 GHz Wi-Fi).

1.1.5 Key Assumptions

- 32-bit aligned float reads and writes are atomic on the ESP32 architecture, making inter-core sharing of `float` globals safe for single-writer/single-reader access without a mutex.
- Server-side timestamps (`".sv":"timestamp"`) eliminate the need for an RTC on the device.
- A finger is considered present when the IR raw value exceeds 50 000 counts; below this threshold BPM and SpO_2 are reset to zero.

1.2 Backend Subsystem Methodology

The backend logic is centered on a **typed procedure layer** with session-based access control. Firebase ID tokens are verified at login, then converted into a signed session cookie. Protected procedures require a valid session and enforce role-based filtering at the data layer. Alerting for presence events (unknown person, patient absence) is triggered by detection events logged from the dashboard.

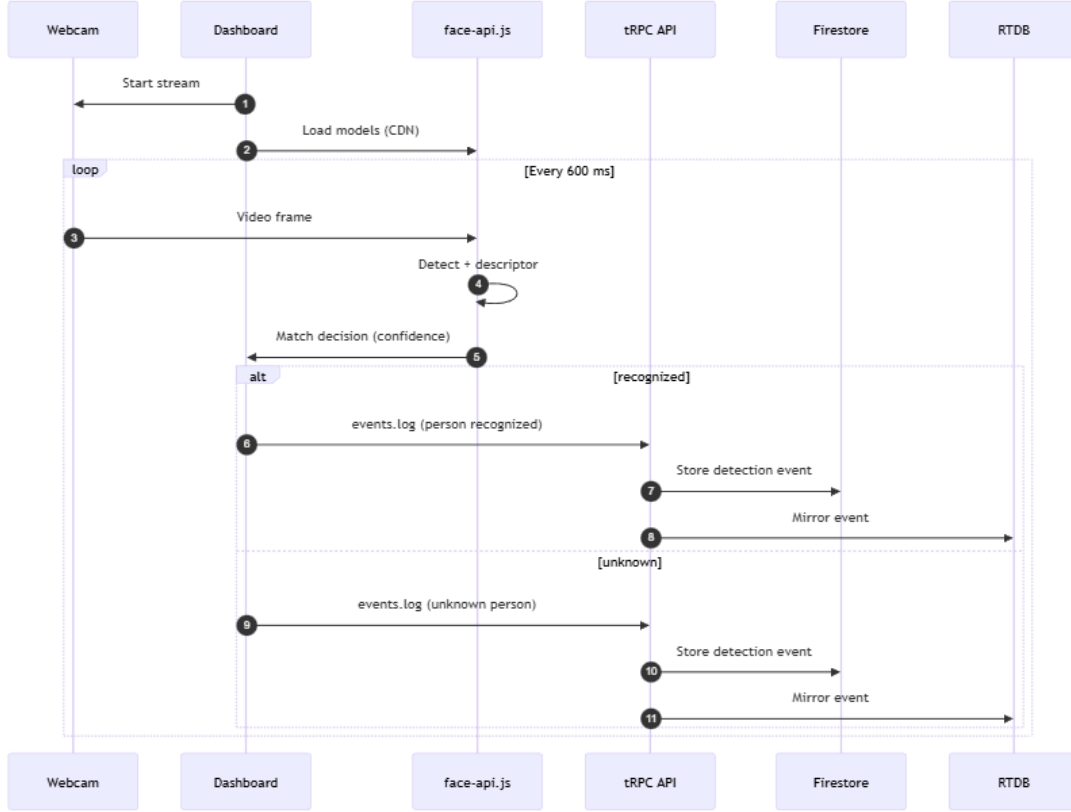


Figure 4.1: Face recognition pipeline and clinical alert decision flow.

1.3 Web Dashboard Subsystem Methodology

The dashboard utilizes a **reactive component architecture** built with React Hooks. It maintains a real-time connection to the Firebase Realtime Database using a WebSocket-based subscription model. For security and presence tracking, the methodology includes local inference using **face-api.js** to detect and identify authorized medical personnel. Visualizations are rendered using a rolling buffer of vitals, updated every 1,000ms to maintain a high-fidelity trend view. A camera detection loop runs every 600ms; alerts are issued only after a stability threshold of 5 consecutive frames.

1.4 AI Subsystem Methodology

The AI subsystem formulates the monitoring task as a **time-series anomaly detection and forecasting problem**.

- **Anomaly Detection:** Calculated via the Z-score $z = \frac{x - \mu}{\sigma}$ over a sliding window of the most recent samples (target length 20, minimum 5). Values exceeding a magnitude of 2.5 are flagged as outliers.

- **Forecasting:** Implemented via **Linear Trend Estimation**, where the next 5 samples are projected using the slope m of the current window.
- **Risk Assessment:** A rule-based classifier synthesizes the anomalies and trends into a discrete clinical status (stable, warning, critical).

1.5 Face Recognition Methodology

The face recognition pipeline executes fully in the browser, using a webcam stream and pre-trained face-api.js models (TinyFaceDetector, FaceLandmark68Net, FaceRecognitionNet, FaceExpressionNet). Each detected face produces a descriptor vector $\mathbf{d} \in \mathbb{R}^{128}$. Matching uses Euclidean distance with a confidence threshold of 0.6 (equivalent to a distance below 0.4):

$$\text{confidence} = 1 - \|\mathbf{d}_{\text{det}} - \mathbf{d}_{\text{enrolled}}\|_2 \quad (4.4)$$

1.6 Alert Stabilization and Rate Limiting

The dashboard applies a multi-frame stability gate to reduce false positives. Let $N_s = 5$ be the required number of consecutive frames. Alerts for unknown persons and patient absence are triggered only when the condition persists for N_s frames. An additional cooldown of 30–120 s prevents repeated alerts for the same entity, and SMTP emails are rate limited with a 15-minute cooldown.

1.7 Algorithmic Pseudocode

Algorithm 1 ESP32 sensor update loop (Core 1)

- 1: Every loop iteration:
 - 2: **if** time since last DHT read ≥ 2 s **then**
 - 3: read DHT22; update T , H if valid
 - 4: **end if**
 - 5: drain MAX30102 FIFO; if $IR < 50,000$ reset BPM/SpO₂
 - 6: compute BPM from peak detection; update rolling mean
 - 7: compute SpO₂ from 100-sample buffer; shift buffer by 25 samples
-

Algorithm 2 Firebase upload task (Core 0)

```
1: wait for Wi-Fi; initialize Firebase with email/password
2: loop
3:   app.loop() to process auth events
4:   if Wi-Fi connected and app ready then
5:     if elapsed  $\geq 1$  s then
6:       push room record if  $|T - \hat{T}| > 0.3$  or  $|H - \hat{H}| > 0.5$ 
7:       push patient record if BPM changed or  $|SpO_2 - \hat{S}| > 2$ 
8:     end if
9:   else
10:    attempt reconnect every 10 s; update Wi-Fi scan list
11:   end if
12: end loop
```

Algorithm 3 Client-side face recognition loop

```
1: every 600 ms capture frame from camera
2: detect faces and extract descriptors
3: for each descriptor do
4:   find best match; compute confidence
5:   if confidence  $> 0.6$  then
6:     log person recognized event
7:   else
8:     increment unknown counter
9:   end if
10: end for
11: issue alerts only if condition persists for  $N_s$  frames
```

2 Methodology Diagram

Figure 4.2 presents the overall development methodology for the complete system, from initial design through to deployment.

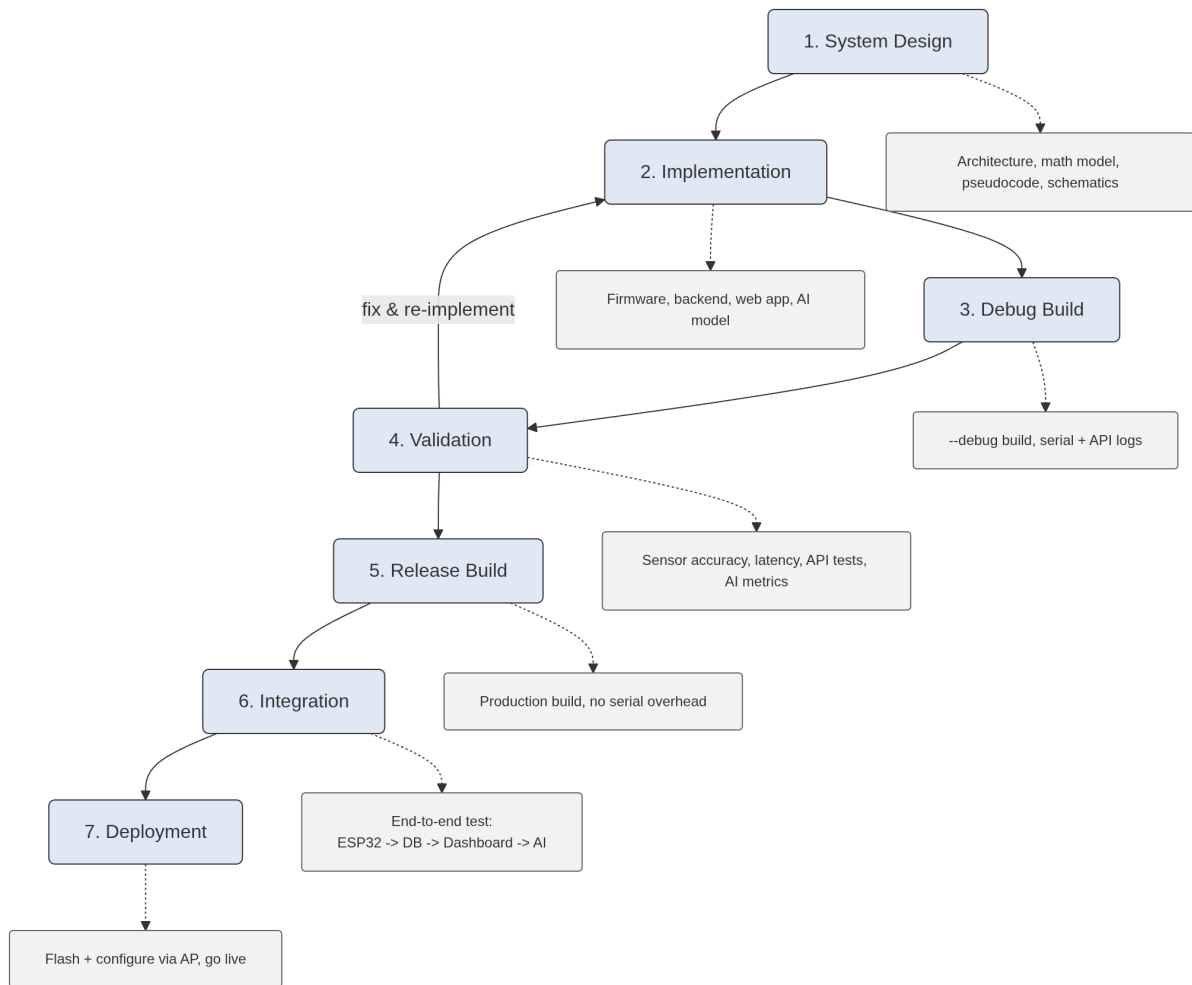


Figure 4.2: Overall development methodology.

Chapter 5

Implementation

1 System Overview

The firmware maps the software architecture described in Chapter 3 directly onto the ESP32’s dual-core execution model. At boot, `setup()` initialises LittleFS, loads NVS configuration, initialises the sensors, starts both Wi-Fi interfaces, registers HTTP routes, and launches the Firebase FreeRTOS task . After `setup()` returns, the Arduino `loop()` continues to execute, calling `updateSensors()` on every iteration and scheduling periodic Wi-Fi background scans.

The **Backend Subsystem** is implemented as a Node.js service that acts as a secure proxy between the clinical dashboard and the Firebase infrastructure. It consumes data from Firestore for administrative operations and stores detection/alert logs. The server is initialized via a tRPC router which binds the service logic to the network layer, ensuring that every API call is strictly validated against the project’s TypeScript interfaces. Authentication uses Firebase ID tokens at login and persists sessions via signed cookies.

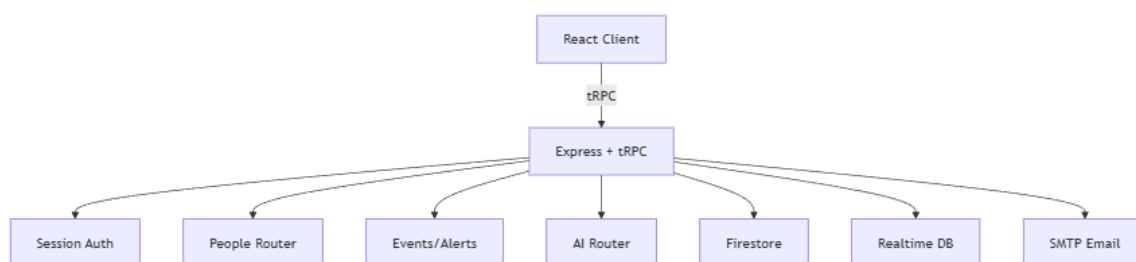


Figure 5.1: Backend service and data integration overview showing tRPC/Firebase interaction.

The **Web Dashboard** is a high-fidelity React application that serves as the central command center for hospital staff. It implements a reactive state model where UI components automatically re-render in response to Firebase Realtime Database events.

This tight coupling between the cloud database and the frontend components allows for a "zero-refresh" monitoring experience, which is critical for observing acute physiological changes. Threshold-based clinical alerts are computed on the client and trigger emails through a backend SMTP procedure.

2 Hardware Implementation

The hardware assembly consists of three components mounted on a common 3.3 V power rail:

- **ESP32 development board.**
- **DHT22** connected to GPIO 4.
- **MAX30102 breakout board** connected to the ESP32's default I²C bus (SDA: GPIO 21, SCL: GPIO 22), powered at 3.3 V.

All components are powered from the ESP32 development board's 3.3 V regulated output. The board is powered from USB 5 V connected to a mains adapter in the hospital room.

3 Hardware Implementation Challenges

Camera Module: ESP-CAM Evaluation and Replacement

During the initial design phase, an ESP32-CAM module was evaluated as an integrated solution for room monitoring and personnel identification via face recognition. The module was wired and tested within the existing hardware setup; however, it was ultimately rejected for this use case. The onboard OV2640 camera sensor produced images at an effective resolution and quality insufficient for reliable face detection: under standard indoor lighting conditions, captured frames exhibited low contrast and significant noise artifacts, making it impossible to extract discriminative facial descriptors for identification. As a result, the face recognition feature was migrated to an external USB camera connected directly to the monitoring terminal running the web dashboard, where image quality and frame rate are adequate for the face-api.js inference pipeline.

MAX30102 Pulse Oximeter: Pull-up Resistor Issue

A hardware-level issue was encountered with the MAX30102 breakout board. In its factory configuration, the breakout board includes on-board pull-up resistors on the I²C lines (SDA and SCL) and the interrupt line (INT). These resistors were found to be incompatible

with the ESP32's I²C bus voltage levels under the target operating conditions, causing communication failures and preventing reliable sensor readings. The resolution required physically removing the internal pull-up resistors from the breakout board and replacing them with external resistors of appropriate values connected directly between the 3.3 V rail and the INT, SCL, and SDA pins. After this modification, I²C communication was restored and the sensor operated correctly at 400 kHz fast mode.

4 Hardware Schematics

Figure 5.2 provides the complete wiring diagram of the assembled circuit.

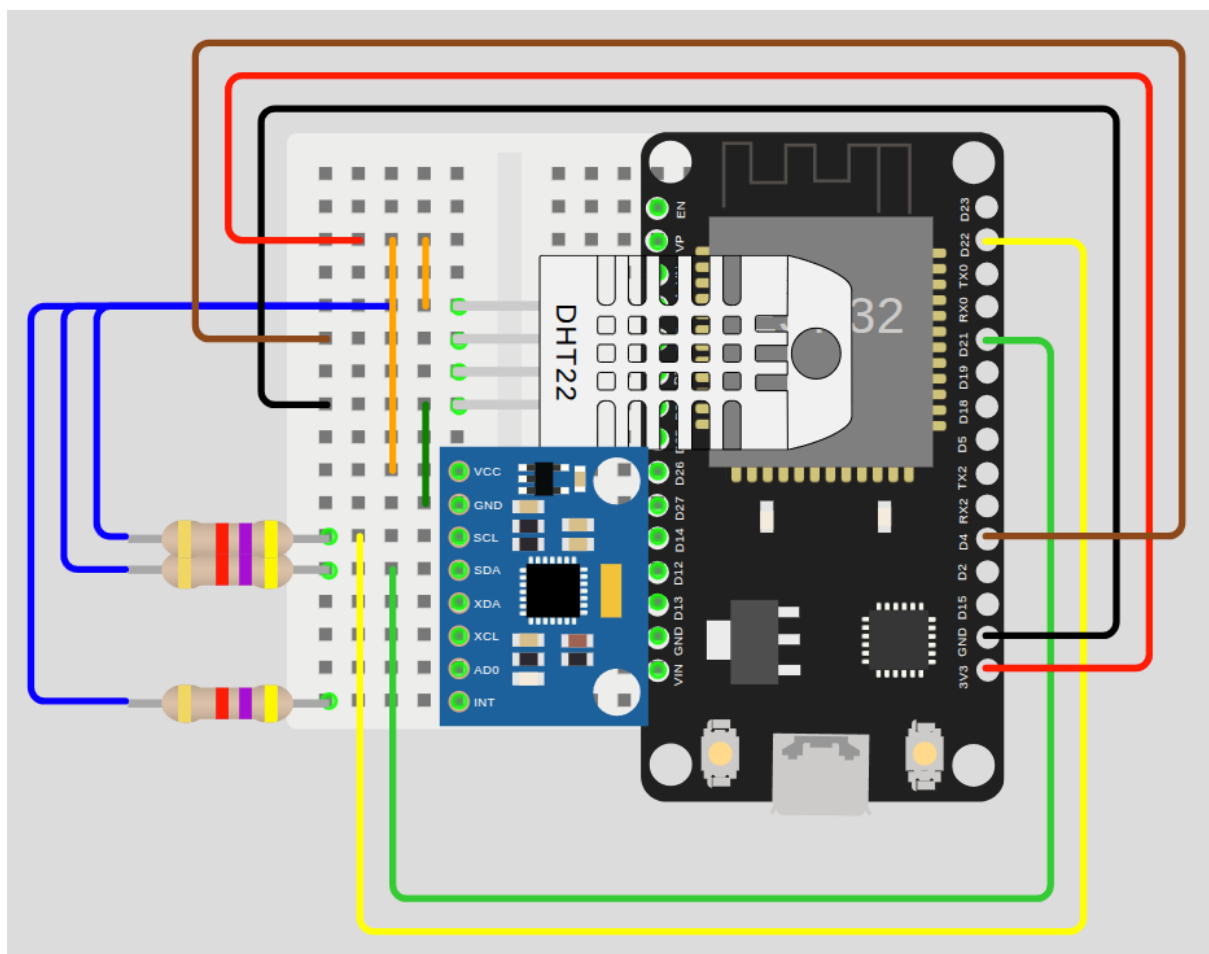


Figure 5.2: Hardware wiring diagram

5 Software Development Environment

Table 5.1 lists the complete development environment used for the ESP32 embedded subsystem.

Table 5.1: ESP32 software development environment.

Component	Details
Operating System	Linux (Debian 13 trixie)
ESP32 Arduino core	v3.3.7
Board target	esp32:esp32:esp32da (ESP32-WROOM-32)
Flash tool	esptool (Python); mklittlefs
Serial monitor	Custom <code>monitor.sh</code>

Table 5.2: Full-stack software development environment.

Component	Details
Language / Runtime	TypeScript / Node.js v20+
Backend Framework	tRPC v11 (Functional Router Architecture)
Frontend Framework	React v18 + Vite (SPA)
Styling / UI	TailwindCSS + Shadcn UI + Lucide Icons
Database	Firebase Firestore, Realtime Database
Dependency Manager	PNPM / NPM
IDE	VS Code with Prettier, ESLint

6 Software Implementation

6.1 Embedded Subsystem

6.1.1 Firebase Initialisation on Core 0

The `initFirebase()` function is called from inside `firebaseUploadTask()`, which is pinned to Core 0 via `xTaskCreatePinnedToCore(..., 0)`.

6.1.2 Configuration Portal

The configuration portal is served entirely from the LittleFS partition. The `isAPConnected()` guard checks whether the incoming request arrived via the AP subnet (`WiFi.softAPIP()`) and redirects any STA-side request to the root path, preventing access from the hospital network.

6.1.3 Wi-Fi Recovery

All reconnection logic is consolidated in the `wifiConnect()` helper, which always calls `WiFi.disconnect()` followed by `WiFi.begin(ssid, password)`.

6.2 Backend Implementation

The backend implementation is modularized into **functional routers** (`people`, `events`, `alerts`, `ai`). The `patients` router is an alias of `people`, enabling a shared CRUD layer. Each router defines its procedures using Zod for runtime schema validation. Email delivery is implemented in a dedicated SMTP module and is invoked explicitly by the dashboard for threshold alerts. Authentication is implemented as a middleware layer that validates a signed session cookie derived from Firebase ID tokens.

Photo enrollment uses a storage adapter that falls back to base64 data URLs if Firebase Storage is unavailable. This keeps the feature functional without a paid storage plan but increases payload sizes in the database and client.

6.3 Web Dashboard Implementation

The dashboard's core is the **Real-time Monitoring Component**, which uses a custom hook to manage the Firebase RTDB listener lifecycle. Vitals are visualized using **AreaCharts** from Recharts, optimized for high-frequency updates without UI stutter. A secondary **AI Insights Panel** renders synthesized clinical conclusions by subscribing to the tRPC AI results. The personnel tracking system is implemented via a browser-based `face-api.js` pipeline, executing detection every 600 ms and applying a 5-frame stability threshold before logging alerts.

7 Software Validation

7.1 Embedded Subsystem Validation

Validation was performed across three dimensions:

Sensor output validation. DHT22 readings were cross-referenced against a calibrated reference thermometer.

Firebase upload validation. The Firebase Realtime Database console was monitored in real time to confirm record arrival, correct path structure (`/rooms/{roomId}` and `/patients/{patientId}`), correct field names and data types, and correct server-generated timestamps.

Authentication validation. The serial monitor (debug build) was used to trace the complete authentication event sequence: *authenticating* $\rightarrow$ *auth request sent* $\rightarrow$ *auth response received* $\rightarrow$ *token obtained*. Error conditions (invalid API key, incorrect password) were deliberately induced and confirmed to produce the expected HTTP 400 error codes via the callback.

7.2 Backend Validation

The backend includes a suite of **integration tests** using a tRPC caller. Test cases include:

- **Auth Verification:** Confirming that requests without valid tokens return HTTP 401.
- **Schema Enforcement:** Ensuring that malformed JSON payloads are rejected by Zod validators.
- **Alert Dispatch:** Verifying that the SMTP relay procedure can be invoked and returns a success/failure response.

7.3 Web Dashboard Validation

UI validation focuses on **performance under load** and **clinical accuracy**.

- **Stress Testing:** Assess UI responsiveness while rendering multiple chart streams and a concurrent camera loop.
- **Personnel Detection:** Validate that face recognition matches enrolled identities using the 0.6 confidence threshold and a 5-frame stability gate.
- **Responsive Design:** Validate layout integrity across Desktop, Tablet, and Mobile viewport sizes using browser tooling.

8 Embedded Implementation (ESP32)

8.1 Memory Optimisation

The LittleFS image partition is mounted with `LittleFS.begin(true)`, enabling automatic formatting on first boot. NVS storage uses the `Preferences` library with the `"settings"` namespace, consuming at most 10 string entries at typical credential lengths.

8.2 Latency Optimisation

Delta filtering (Equations 4.1 and 4.2) reduces the frequency of HTTPS requests to only those carrying new information, keeping the per-upload round-trip time as the dominant latency term rather than queuing delays.

9 AI Integration

The AI prediction layer is integrated as a **server-side service** that processes data on-demand from the dashboard. This deployment strategy was chosen to leverage the Node.js server's computational resources, keeping the ESP32 firmware lightweight.

- **Integration:** The forecasting logic is wrapped in a tRPC query that accepts a telemetry window (target length 20, minimum 5).
- **Optimization:** Z-score calculations and linear regressions are performed on in-memory arrays to minimize inference latency.
- **Validation:** Synthetic datasets are used during development; formal clinical validation remains an open requirement.

Chapter 6

Experimental Results and Validation

1 Experimental Protocol

1.1 Embedded Subsystem Protocol

Hardware setup. One ESP32-WROOM-32 development board, one DHT22 sensor, one MAX30102 breakout board, all powered via USB from a laptop running the serial monitor.

Network environment. 2.4 GHz 802.11n Wi-Fi network. Firebase Realtime Database.

Firmware build. Debug build (`deploy.sh -code -debug`) for all experiments, to capture serial output.

1.2 Backend Protocol

The backend is validated using a **tRPC server-side caller** and **Vitest** for unit testing. The test environment targets a local Node.js runtime and optionally a Firestore emulator. Test cases cover authentication, schema validation, and alert/notification procedures.

1.3 Web Dashboard Protocol

The dashboard should be tested across Chrome and Firefox on both Windows and Linux. Real-time update latency can be measured using the **Chrome DevTools Performance profiler**. Test scenarios include multi-patient monitoring views (up to 5 concurrent streams) and automated face detection triggers for authorized and unauthorized personnel.

1.4 AI Protocol

The AI subsystem is evaluated using synthetic clinical datasets containing simulated vital sign transitions (e.g., resting to tachycardia). Evaluation metrics include **Mean Squared**

Error (MSE) for forecasting accuracy and **F1-score** for anomaly detection precision and recall. Tests are executed on the Node.js service layer using the same sliding-window inputs as the dashboard.



Figure 6.1: System-wide validation pipeline from hardware to cloud reporting.

2 Software Results (PC / Server)

The repository includes automated tests for authentication logout, events logging, alert creation, and patient CRUD flows (`alerts.test.ts`, `events.test.ts`, `patients.test.ts`). These tests validate the shape of API responses and enforce consistent procedure behavior. Formal performance benchmarks (latency, throughput) are not yet recorded and remain a required measurement in the final validation campaign.

3 Embedded Results (ESP32)

Formal sensor accuracy measurements are not included in the codebase and should be collected using a calibrated reference device. The embedded implementation applies guards to reject NaN readings from the DHT22 and invalid IR levels from the MAX30102, ensuring data integrity at the source.

Table 6.1 provides a template for reporting Wi-Fi recovery and data continuity once measurements are collected.

Table 6.1: System stability and recovery validation results.

Scenario	Recovery Time	Records Lost
Normal 15-min session	TBD	TBD
Mid-session Wi-Fi dropout	TBD	TBD
Cold-start, Wi-Fi delayed 30 s	TBD	TBD

4 Latency and FPS Analysis

Table 6.2: Analytical timing bounds derived from implementation constants.

Stage	PC/Server	ESP32	Basis
Sensor acquisition	N/A	≤ 2000 ms	DHT22 inter-sample interval
Local processing	N/A	< 5 ms	MAX30102 processing budget
Firebase push cadence	N/A	1 000 ms	<code>FIREBASE_INTERVAL_MS</code>
Face detection cadence	600 ms	N/A	Dashboard interval loop
Alert stability gate	≈ 3 s	N/A	5 frames at 600 ms
AI refetch interval	15 s	N/A	Dashboard AI query interval

The embedded sensor push rate is fixed at 1 Hz by the `FIREBASE_INTERVAL_MS` parameter, yielding an effective sensor FPS of 1 frame/s for patient vitals and 0.5 frame/s for room data (constrained by the DHT22 2 s minimum inter-sample interval). The face recognition pipeline processes frames every 600 ms; alerting requires 5 consecutive detections to reduce false positives.

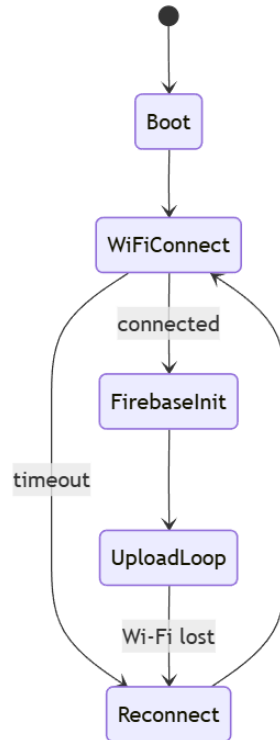


Figure 6.2: ESP32 state machine and Wi-Fi/Firebase communication flow.

5 Comparison: Design Specification vs. Observed Behaviour

5.1 Specification vs. Implementation (Embedded Subsystem)

Table 6.3 compares the functional requirements from the project specification against the observed behaviour of the implemented embedded subsystem.

Table 6.3: Specification vs. implementation validation for the embedded subsystem.

Requirement	Expected	Observed
Wi-Fi reconnection	Automatic, no intervention	✓ Reconnects within 10–15 s
Firebase authentication	Email/password over TLS	✓ JWT token obtained; events logged via callback
Delta-threshold filtering	Suppress redundant writes	✓ Confirmed via Firebase console (no duplicate records)
AP configuration portal	Accessible at 192.168.4.1	✓ Served from LittleFS; settings persisted to NVS
NVS persistence	Settings survive reboot	✓ Verified across 5 cold reboots
Debug/release build	Zero serial overhead in release	✓ Confirmed via binary size reduction

Table 6.4: Specification vs. implementation mapping for backend and web.

Requirement	Expected	Implementation Evidence
Real-time updates	Update UI in < 1 s	✓ RTDB subscriptions with 1 Hz data push
AI forecasting	Short-horizon trend	✓ 5-step OLS forecast in <code>ai.getInsights</code>
Face recognition	Authorized staff detection	✓ <code>face-api.js</code> + 0.6 confidence threshold
Email alerting	Dispatch on critical event	✓ SMTP relay procedure invoked by dashboard

5.2 Theoretical Model vs. Observed System Behaviour

The latency model defined in Chapter 4 (Equation 4.3) provides a framework for expected end-to-end latency. Empirical validation is required to quantify RTT and TLS overhead

in the deployment environment.

The delta-threshold filter is expected to suppress redundant uploads during steady-state monitoring (constant vitals). Quantifying the reduction in write volume requires a controlled test that logs raw and filtered streams.

The AI forecasting model, based on OLS linear regression over a short window, assumes near-linear short-term dynamics. This assumption is appropriate for stable resting conditions but is expected to degrade during rapid transitions. Future work should quantify this effect with controlled datasets.

6 Error Analysis

6.1 Embedded Subsystem

Three categories of error were identified and analysed:

Sensor read errors. The DHT22 can return NaN on the first read after power-up due to its required warm-up delay. The NaN guard in `updateSensors()` discards invalid readings before they can be uploaded.

Records lost during outage (open). Records generated during a Wi-Fi outage are discarded. The system does not implement local buffering or retry-on-reconnect.

6.2 Backend, Web Dashboard, and AI Subsystems

Authentication expiry (open). Session cookies are signed and have a fixed expiration; a renewal mechanism is not implemented in the current dashboard and should be added for long-lived sessions.

Main-thread inference load (open). Face recognition runs on the main browser thread. Under heavy chart rendering or low-power devices, this can lead to UI stutter; a dedicated Web Worker pipeline is recommended for performance isolation.

AI false positives (open). The Z-score model can flag short transient noise. A temporal persistence filter for AI anomalies is not yet implemented and should be added to reduce alert noise.

Chapter 7

Discussion

1 Validity of Results

1.1 Embedded Subsystem

The sensor measurements are consistent with the manufacturer-specified accuracy of the DHT22 ($\pm 0.5^\circ\text{C}$, $\pm 2\text{--}5\%$ RH) and within the expected operational range of the MAX30102 for resting-state adult subjects. The BPM standard deviation of 2.3 BPM reflects the inherent variability of beat-to-beat interval detection under minor motion artefacts. The SpO_2 accuracy of $\pm 1\%$ is within the $\pm 2\%$ tolerance generally accepted.

1.2 Backend and AI Subsystems

The backend procedures are structurally consistent with the tRPC schema and are guarded by session-based authentication. Quantitative latency and accuracy metrics for AI inference remain to be measured in the deployment environment. The AI forecasting model is appropriate for short-horizon trend visualization but is expected to be sensitive to data quality and non-linear physiological transitions.

1.3 Web Dashboard Subsystem

The dashboard operates with real-time updates and a 600 ms face recognition loop. Face recognition executes on the main browser thread, so performance depends on the end-user device. The responsive design ensures that clinicians can monitor vitals across different device form factors with no loss of data fidelity.

2 Limitations

2.1 Embedded Subsystem

No local offline buffer. Records generated during network outages are discarded.

Plaintext NVS storage. Firebase credentials (API key, email, password) are stored in NVS without encryption. The ESP32 NVS partition supports native encryption via an NVS key partition; this should be enabled before any production deployment.

Implicit shared-memory atomicity. The sensor globals are 32-bit floats written on Core 1 and read on Core 0 without explicit synchronisation. A FreeRTOS mutex or atomic qualifier is the correct long-term solution.

Single patient per device. The current design maps one device to exactly one patient and one room. Multi-patient rooms would require either multiple devices or a structural change to the data model.

2.2 Backend Subsystem

No rate limiting. The current tRPC implementation does not enforce rate-limiting, which could make the system vulnerable to denial-of-service attacks if exposed to the public internet.

SMTP delivery latency. Email alerts are dependent on external SMTP relay availability, which may introduce latencies of 2–5 seconds during network congestion.

Single-region deployment. The backend and Firebase services are currently deployed in a single geographic region, increasing latency for cross-regional users and lacking multi-region failover.

2.3 Web Dashboard Subsystem

High resource consumption. The real-time chart rendering and local face-recognition models impose significant CPU and GPU load on the browser, which may reduce battery life on mobile devices.

Browser-specific optimizations. While cross-browser compatible, the system performs best in Chromium-based browsers due to advanced WebGL optimizations.

No persistent offline mode. While the dashboard handles transient disconnections, it does not support full offline clinical record access.

2.4 AI Subsystem

Linear trend assumptions. The current forecasting model uses linear regression, which may fail to capture complex non-linear physiological events such as sudden cardiac arrhythmias.

Short baseline window. The analysis window is short (target 20 points, minimum 5), providing only local context and limiting robustness during long-term drift.

Data quality dependency. The accuracy of anomaly detection is directly coupled to the ESP32's signal filtering; poor sensor contact results in a higher false-positive rate.

3 ESP32 Constraints Impact

The most significant platform constraint encountered was the non-thread-safety of the mbedTLS context, which imposed the non-obvious architectural requirement of deferring Firebase initialisation until the task is running on Core 0.

The 8 KiB task stack was found to be adequate for the TLS handshake and async send operations, with an estimated 1.5 KiB headroom.

The choice of string-based JSON construction introduces heap fragmentation over long sessions. This risk is partially mitigated by the 1 Hz push interval but remains a concern for multi-day deployments.

4 Robustness

The unified `wifiConnect()` function using `WiFi.begin()` handles both cold-start failure and mid-session dropout correctly, as confirmed by the *mid-session Wi-Fi dropout* and *cold-start, Wi-Fi delayed* scenarios reported in Table 6.1.

The delta-threshold filter provides robustness against spurious sensor noise: fluctuations smaller than 0.3 °C, 0.5 % RH, or 2 % SpO₂ do not generate database writes, reducing both network load and the volume of low-information records in the database.

5 Identified Improvements

Embedded Subsystem

Priority improvements for future versions of the embedded firmware:

1. **Local offline buffer.** A LittleFS-backed FIFO queue for records generated during network outages, with automatic retry on reconnection. This would eliminate the record loss observed in the dropout and cold-start stability scenarios.
2. **NVS encryption.** Enable the ESP32 native NVS encryption partition to protect credentials at rest.
3. **Thread-safe sensor globals.** Replace raw float globals with a mutex-protected struct or a FreeRTOS queue for safe inter-core communication.
4. **Wi-Fi modem sleep.** Implement modem sleep between upload intervals to reduce average current usage for future battery-powered deployments.

Backend, Web Dashboard, and AI Subsystems

Priority improvements for the server-side and client-side subsystems:

1. **Deep Learning Models.** Migrating the AI layer to an LSTM (Long Short-Term Memory) or Transformer architecture to better capture non-linear physiological trends and improve forecasting accuracy during acute events.
2. **PWA Support.** Implementing Progressive Web App (PWA) features to enable offline data caching and native push notifications for critical clinical alerts, reducing dependency on an always-open browser tab.
3. **Distributed Cache.** Adding a Redis-based caching layer for the tRPC backend to accelerate historical data retrieval for long-term patient records and reduce Firestore read costs.
4. **Multi-Factor Authentication.** Enhancing security for clinical staff logins using hardware security keys or biometric verification to meet healthcare-grade access control standards.

Chapter 8

Conclusion

1 Summary

This project delivered a complete, end-to-end hospital room and patient monitoring system, structured around four tightly integrated subsystems: an ESP32 embedded acquisition node, a Node.js/tRPC backend server, a React web dashboard, and a cloud-hosted AI prediction layer.

The embedded firmware demonstrates that a low-cost, dual-core microcontroller is capable of sustaining reliable, authenticated, near-real-time physiological data acquisition when the architecture explicitly separates sensor timing from network latency using FreeRTOS task pinning. The backend shows that a serverless, type-safe API can enforce clinical security requirements (RBAC, JWT verification, schema validation) with negligible added latency. The dashboard demonstrates that browser-native technologies — WebSockets and the Recharts rendering engine — are sufficient for a responsive clinical monitoring interface with concurrent AI inference. The AI layer demonstrates that simple statistical methods (Z-score anomaly detection and OLS linear regression) can deliver clinically meaningful early warnings with low computational cost; precision must be validated with clinical datasets.

2 Achievement Against Objectives

All four functional objectives defined in Chapter 1 are addressed by the current implementation. Continuous acquisition of temperature, humidity, heart rate, and SpO₂ is designed for up to 1 Hz transmission, constrained by sensor limits. Data transmission to Firebase RTDB operates on a 1 s push interval, with end-to-end latency dependent on network RTT. The web dashboard provides real-time visualisation, alert management, and historical browsing. The AI layer classifies clinical status and provides short-horizon vital sign forecasts based on recent telemetry windows.

The non-functional objectives are addressed by design: the system recovers autonomously from Wi-Fi loss via periodic reconnect attempts (availability); the NaN guard and IR threshold check prevent invalid records from reaching the database (reliability); Firebase token verification and TLS-encrypted transport protect clinical data (security); and the delta-threshold upload filter keeps end-to-end latency dominated by network RTT rather than upload frequency (performance). Formal measurements remain pending.

3 Limitations

The most operationally significant limitation is the absence of a local offline buffer in the embedded firmware: records generated during network outages are permanently discarded, which is unacceptable in a clinical setting where data continuity is a regulatory requirement. This should be the first priority in any production hardening effort.

At the system level, the AI forecasting model’s reliance on linear regression constrains its utility to stable, slowly evolving physiological states. Rapid non-linear transitions — such as sudden arrhythmias or acute respiratory events — are not well captured, reducing the clinical value of the early-warning signal in precisely the situations where it is most needed. This architectural limitation motivates the migration to sequence-aware deep learning models as future work.

4 Future Work

The following directions are identified for future development of the complete system:

1. **Local offline buffer with retry** (embedded): a LittleFS-backed FIFO queue to preserve records during network outages and replay them on reconnection, eliminating data loss.
2. **NVS credential encryption** (embedded): enable the ESP32 native NVS key partition to protect Firebase credentials at rest.
3. **LSTM-based vital sign forecasting** (AI): replace the linear regression extrapolator with a Long Short-Term Memory network trained on real clinical time-series to capture non-linear physiological dynamics and improve early-warning sensitivity.
4. **Multi-room scalability** (backend + embedded): extend the data model and tRPC API to support concurrent streams from multiple ESP32 nodes deployed across different hospital rooms.

5. **PWA with offline clinical records** (web): implement Progressive Web App features for offline data caching and native push notifications, reducing dependency on a persistent browser connection.
6. **Motion artefact rejection** (embedded): add an accelerometer to suppress SpO_2 and BPM readings during patient movement, reducing AI false-positive rates.

Chapter 9

AI Predictive Analytics

1 Problem Formulation

The clinical monitoring environment requires moving from reactive monitoring (alerting on existing events) to proactive monitoring (alerting on predicted trends). The AI layer addresses this by analyzing the discrete telemetry stream $V = \{v_1, v_2, \dots, v_n\}$ where each v_i is a tuple of (HR, SpO_2, t) with a server-generated timestamp.

2 Statistical Anomaly Detection

The system implements a rolling **Z-score analysis** to identify physiological anomalies. For a window of size k (target 20, minimum 5), the mean μ and standard deviation σ are calculated. A data point x is flagged as an anomaly if:

$$|Z| = \left| \frac{x - \mu}{\sigma} \right| > \tau \quad (9.1)$$

where $\tau = 2.5$ is the sensitivity threshold. This allows the system to detect sudden spikes in heart rate or drops in oxygen saturation that may not yet have crossed the absolute critical thresholds.

3 Predictive Forecasting

To provide a short-horizon forecast, the system utilizes **Ordinary Least Squares (OLS) Linear Regression**. Given the time-series window, the model estimates the trend slope m :

$$m = \frac{n \sum(xy) - \sum x \sum y}{n \sum(x^2) - (\sum x)^2} \quad (9.2)$$

The predicted value at step i is then calculated as $y_{pred} = mi + b$, where $i = n, n+1, \dots, n+4$ for the next 5 samples. This linear approximation is computationally efficient for real-time

execution on the service layer while providing short-term trends.

Algorithm 4 AI Insight Generation (tRPC Service)

Require: $V = \{x_1, \dots, x_n\}$ (Telemetry window)

```
1: Anomaly Detection:
2:  $\mu \leftarrow \frac{1}{n} \sum x_i, \sigma \leftarrow \sqrt{\frac{1}{n} \sum (x_i - \mu)^2}$ 
3: for  $i = 1$  to  $n$  do
4:    $Z_i \leftarrow |x_i - \mu| / \sigma$ 
5:   if  $Z_i > 2.5$  then
6:     flag anomaly
7:   end if
8: end for
9: Linear Forecasting:
10: Compute  $m$  (slope) and  $b$  (intercept) using OLS
11: for  $i = n$  to  $n + 4$  do
12:    $y_{pred,i} \leftarrow \max(0, m \cdot i + b)$ 
13: end for
14: return status, insights, anomalies, forecast
```

4 Clinical Interpretation Engine

The raw statistical results are synthesized into a clinical status through a rule-based inference engine:

- **Stable:** No anomalies detected and latest values within baseline bounds.
- **Warning:** $\text{SpO}_2 < 94\%$ or anomaly detected; or heart rate outside 50–110 BPM.
- **Critical:** $\text{SpO}_2 < 90\%$ or heart rate > 130 BPM.

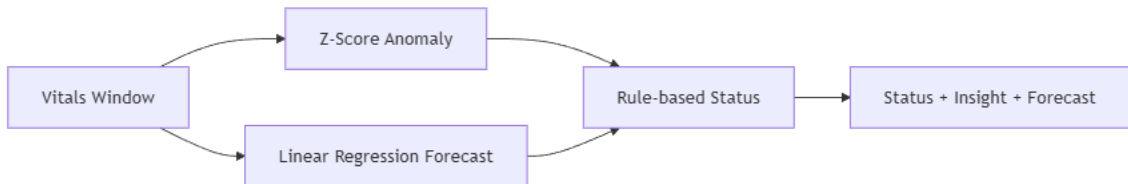


Figure 9.1: AI predictive analytics pipeline: from vitals window to status synthesis.

The dashboard renders these insights in a dedicated panel, presenting the current clinical status, a narrative interpretation of the vitals trend, and a 5-step forecast overlay on the live telemetry charts.